

The state of the art in automated requirements elicitation



Hendrik Meth^{a,*}, Manuel Brhel^b, Alexander Maedche^{a,b}

^a Institute for Enterprise Systems, University of Mannheim, L 15, 1-6, 68131 Mannheim, Germany

^b Chair of Information Systems IV, University of Mannheim, L 15, 1-6, 68131 Mannheim, Germany

ARTICLE INFO

Article history:

Received 12 October 2012

Received in revised form 24 March 2013

Accepted 24 March 2013

Available online 2 April 2013

Keywords:

Requirements Engineering

Requirements Elicitation

Automation

Requirements Reuse

Systematic Review

ABSTRACT

Context: In large software development projects a huge number of unstructured text documents from various stakeholders becomes available and needs to be analyzed and transformed into structured requirements. This elicitation process is known to be time-consuming and error-prone when performed manually by a requirements engineer. Consequently, substantial research has been done to automate the process through a plethora of tools and technologies.

Objective: This paper aims to capture the current state of automated requirements elicitation and derive future research directions by identifying gaps in the existing body of knowledge and through relating existing works to each other. More specifically, we are investigating the following research question: What is the state of the art in research covering tool support for automated requirements elicitation from natural language documents?

Method: A systematic review of the literature in automated requirements elicitation is performed. Identified works are categorized using an analysis framework comprising tool categories, technological concepts and evaluation approaches. Furthermore, the identified papers are related to each other through citation analysis to trace the development of the research field.

Results: We identified, categorized and related 36 relevant publications. Summarizing the observations we made, we propose future research to (1) investigate alternative elicitation paradigms going beyond a pure automation approach (2) compare the effects of different types of knowledge on elicitation results (3) apply comparative evaluation methods and multi-dimensional evaluation measures and (4) strive for a closer integration of research activities across the sub-fields of automatic requirements elicitation.

Conclusion: Through the results of our paper, we intend to contribute to the Requirements Engineering body of knowledge by (1) conceptualizing an analysis framework for works in the area of automated requirements elicitation, going beyond former classifications (2) providing an extensive overview and categorization of existing works in this area (3) formulating concise directions for future research.

© 2013 Elsevier B.V. All rights reserved.

Contents

1. Introduction	1696
2. Analysis framework	1696
2.1. Design perspective	1697
2.1.1. Tool categories	1697
2.1.2. Degree of automation	1698
2.1.3. Reuse of requirements and requirements knowledge	1698
2.2. Evaluation perspective	1698
2.2.1. Evaluation approaches	1698
2.2.2. Evaluation concepts and measures	1698
3. Methodology	1699
4. Results	1700
4.1. Results per tool category	1700

* Corresponding author. Tel.: +49 621 181 3579; fax: +49 621 181 3627.

E-mail addresses: meth@es.uni-mannheim.de (H. Meth), mbrhel@mail.uni-mannheim.de (M. Brhel), maedche@es.uni-mannheim.de (A. Maedche).

4.2.	Degree of automation	1702
4.3.	Reuse of requirements and requirements knowledge.....	1702
4.4.	Evaluation approaches	1703
4.5.	Evaluation concepts and measures	1703
4.6.	Result of the citation analysis.....	1704
5.	Discussion and propositions for future research	1704
5.1.	Degree of automation	1704
5.2.	Reuse of requirements and requirements knowledge.....	1705
5.3.	Evaluation approaches	1705
5.4.	Evaluation concepts and measures	1705
5.5.	Discussion of the citation analysis result.....	1706
6.	Conclusion	1706
	Acknowledgements	1706
	Appendix A.....	1706
	Appendix B.....	1707
	References	1707

1. Introduction

In software development, the elicitation of high quality requirements is a critical success factor [1]. Shortcomings in requirements elicitation can have a negative impact on the overall development process [2], and lead to high costs for the involved organizations [3]. While the correction of shortcomings during requirements elicitation has been reported to account for up to 75% of all error removal costs throughout the software development life cycle [4], the lack of clear and complete requirements can even lead to the complete failure of a software development project [5,6].

According to Mich et al. [7], natural language is the most common and important medium for requirements documents: 79% of all requirements documents are written in common natural language. Natural language is inherently powerful and expressive and can therefore be used to communicate between a broad range of stakeholders and users [8]. It can be used in diverse ways to document requirements, for example within narrative scenarios, support messages, emails, workshop memos or even interview transcripts. Natural language requirements can be formulated in documents as well as other resources (e.g., entries in issue tracking or test case management systems, support databases or discussion forums).¹ Furthermore, they can be combined with more formal requirements representations (e.g., UML models) or figures (e.g., mockups). Although natural language appears to be a well suited mean to articulate and discuss requirements, severe problems emerge when using it in specification documents as they might be ambiguous, inconsistent and incomplete [9]. Furthermore, a direct interpretation of these documents by subsequent development tools is almost impossible [8]. Therefore natural language requirements are usually transformed from initially informal statements into more consistent and unambiguous models² [10].

Especially in large development projects, this is a challenging task as a huge number of natural language documents become available and need to be analyzed. In these cases, manual requirements elicitation can become time-consuming, error-prone, and monotonous, especially if it is repeated multiple times, whenever updates to previously existing documents become available [11,12]. Given the fact that requirements engineers are scarce, valuable resources, a more efficient way to support their work seems imperative. In a study about current requirements elicitation practices, Mich et al. [7] asked more than 150 participants

working in software development roles³ to name the two things in their job they would like to do more efficiently. The activity that was named most frequently (46%) was “identify user requirements”.

Consequently, much effort has been devoted to leverage tools and technologies enhancing requirements elicitation [13,14]. Still, there seems to be vast potential for improvement. Although calls for automated support of Requirements Engineering (RE) processes have already been made almost 30 years ago [15], “automation” is still on top of most software developers’ wish lists: in the survey mentioned above, 69% of the respondents identified automation as the most valuable contribution to an improvement of RE practices [7].⁴

The work presented in this paper therefore aims to conceptualize an analysis framework for works in the area of automated requirements elicitation, capture the current state of this area and outline future research avenues by identifying gaps in the existing body of knowledge. More specifically, we are investigating the following research question: *What is the state of the art in research covering tool support for automated requirements elicitation from natural language documents?* We detail this overall question by investigating the identified works along different dimensions and analyze them both from a design as well as an evaluation perspective. Taking these two perspectives allows for a holistic assessment and categorization of former works covering the two main phases of tool-oriented research: the design of the tool and its evaluation.

The remainder of this paper is structured as follows: In Section 2 the framework used for the literature analysis is conceptualized. In Section 3, an outline of the methodological approach to perform the literature analysis is given. The results of the literature analysis are presented in Section 4, while Section 5 gives a summary of the findings and future research opportunities, followed by final conclusions in Section 6.

2. Analysis framework

Pohl [16] defines requirements elicitation as a core RE activity aiming to (1) determine relevant requirements sources (2) identify requirements from these sources and (3) develop new requirements. Automated requirements elicitation primarily focuses the second activity, which is also our focus in this paper. As stated in the research question, we want to capture the state of the art in tool support for automated requirements elicitation from natural

¹ In the following, we refer to these different types of representations as “requirements documents” for simplicity.

² The selection of a fitting model type depends on the specific business and application context.

³ According to Mich et al. [7], participants worked in different roles in the software development process. While the majority of European respondents selected “System Engineer/Architect”, most American participants selected “Project Manager”, which may indicate that there are regional differences in the naming of specific roles.

⁴ RE = Requirements Engineering. NL = Natural Language.

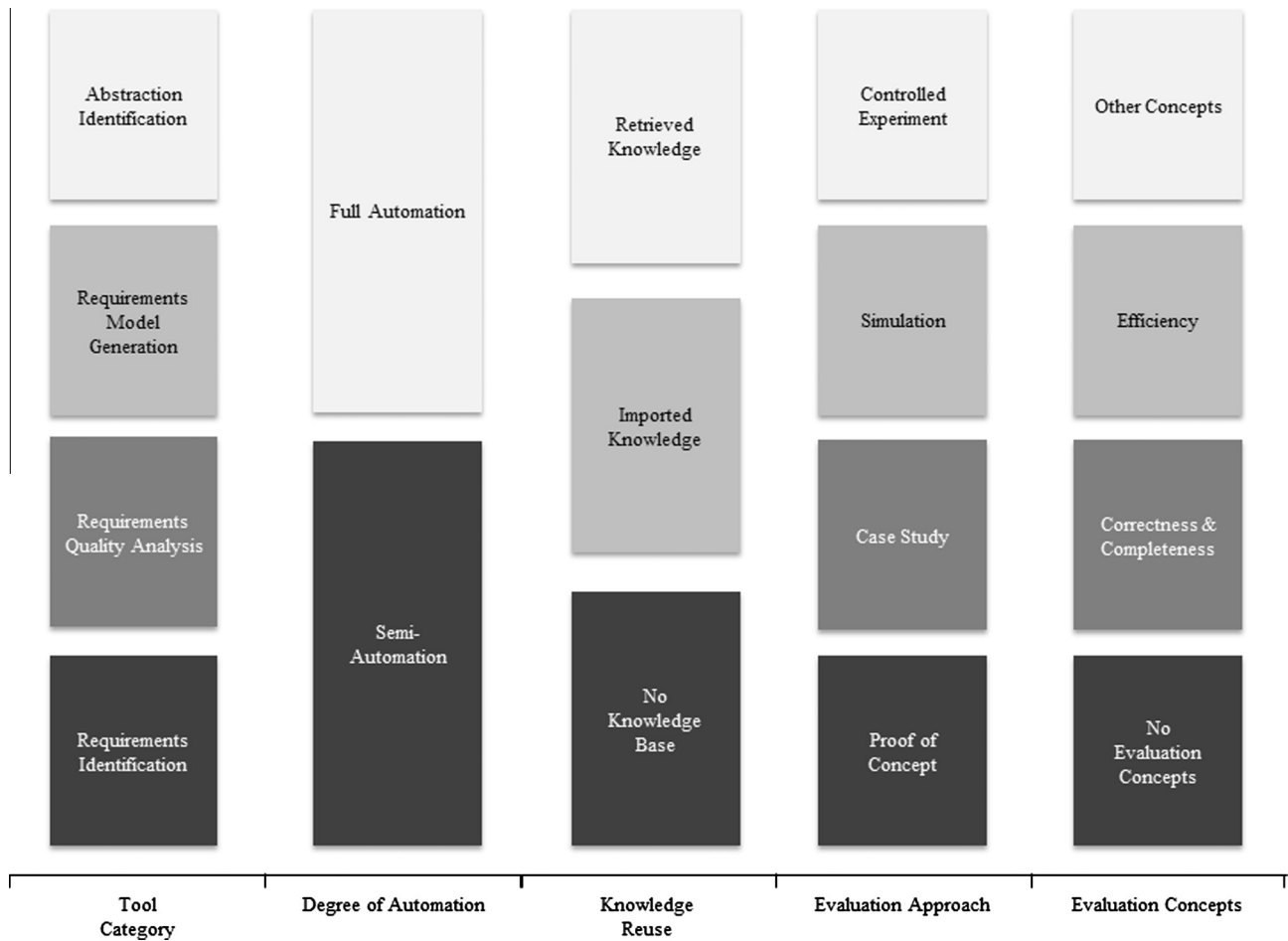


Fig. 1. Literature analysis framework.

language documents. This current state is observed from two perspectives. First, taking a design perspective, we are describing the underlying technological concepts of state of the art tool support. Second, taking an evaluation perspective we are depicting how the effectiveness of these tools has been evaluated. We are hereby breaking down the initial research question into two sub-questions: (1) *Which tool categories and technological concepts are employed in automated requirements elicitation?* (2) *Which concepts and approaches have been applied to evaluate tool effectiveness?* The mentioned tool categories, technological concepts and evaluation concepts relevant for this analysis are representing the categories of our analysis framework which is depicted in Fig. 1. In the following, this framework will be introduced in detail.

2.1. Design perspective

In the design perspective, different tool categories and technological concepts are differentiated.

2.1.1. Tool categories

Berry et al. [17] introduce different categories of tools to support the processing of natural language requirements with full or semi-automation. Three of these categories are of relevance in the context of requirements elicitation⁵:

- (1) *Abstraction identification*: Tools that identify the key abstractions from a natural language document, e.g. to assist an analyst in gaining an understanding of an unfamiliar domain. Abstractions represent the main concepts and most significant terms which can serve as an initial list or ontology of requirements to be used for negotiation with the customer. These tools can be seen as an early step in requirements elicitation, which is necessary to build up relevant knowledge. The generated knowledge can then be used for the analysis of further documents in the following. An example is the AbstFinder tool presented by Goldin and Berry [18].
- (2) *Requirements model generation*: Tools to generate models from NL descriptions. This group of tools addresses the core of the requirements elicitation process, supporting the identification and classification of requirements as well as their transformation to abstract models. An example is the work by Popescu et al. [19].
- (3) *Requirements quality analysis*: Tools to find defects or deviations from good requirements practice in NL requirements documents. These tools focus on the analysis of existing requirements rather than the initial detection of requirements in NL documents, but will be included in the analysis if a relationship to this problem is given by the respective authors. The QuARS tool by Fabbrini et al. [20] is an example for this category.

In addition to these three categories, we identified a fourth relevant category during our analysis, which will be already introduced at this point for completeness.

⁵ The authors mention a fourth category of tools, which is out of scope for requirements elicitation. Tools of this category aim to infer trace links between natural language descriptions of requirements or between requirements and other artifacts of the development process.

- (4) *Requirements identification*: These tools can be positioned between the first and second category, aiming directly at the elicitation of structured requirements from unstructured or semi-structured sources of information. Building on abstractions and knowledge, which can be created by abstraction identification tools, these systems do not generate models yet but identify single requirements statements. For example, Cleland-Huang et al. [21] apply information retrieval methods to find and classify non-functional requirements.

2.1.2. Degree of automation

Following our research question, automation represents an obvious technological concept of the tools under investigation. However, existing works show differences concerning the degree of automation provided. Research suggests that while tool support can cause an efficiency advantage in comparison to a purely manual analysis [14], a complete automation of RE tasks can lead to a loss of information or erroneous results [18]. Berry [22] points out that the cognitive aspects of requirements elicitation should not be underestimated, as tools may omit important requirements, and fail to detect logically correct, but questionable, requirements. Thus, automation approaches should “bring the human into the loop and promote thinking about the results” [22]. This indicates a conflict between the benefits of automation and the necessity of human intervention.

According to Parasuraman et al. [23], the appropriate degree of automation in the support of human tasks should be chosen according to a variety of evaluative criteria, including the reliability of the automation and the costs of decision outcomes. While a *full automation* would replace the human analyst, a *semi-automated* approach would merely support him and thus rationalize requirements elicitation, while still requiring an interaction with the system. In contrast to (semi-)automatic approaches, during manual requirements elicitation (which is not within the scope of this literature review⁶) an analyst would start the analysis from scratch, without any potential requirements recommended by the system. This said, it should be noted that in practice the degree of automation should rather be seen as a continuum than a categorical concept. While a fully automated approach might target to replace any manual elicitation activity, in most cases an analyst will still double-check at least parts of the results of the automatism to make sure that requirements have been captured correctly. In this sense the differentiation between semi-automation and full automation which will be made in this review should be rather understood as the design and usage focus of a tool.

2.1.3. Reuse of requirements and requirements knowledge

A technological concept closely related to automated elicitation is the reuse of existing requirements or knowledge [11,21,24]. Reuse is defined as the assembly of a product from existing components, while the creation of new components is limited to ones that do not exist [25]. It is a popular concept in software engineering, where it usually applies to the adaption of existing source code, and the use of libraries or software components [26]. Different approaches to requirements reuse have been proposed in the literature, most of which rely on tool support and are thus in a close relationship to the automated methods presented above. The RARE (Reuse-Assisted Requirements Engineering) method presented by Cybulski and Reed [27] supports the identification and replacement of new requirements with those available in existing and already processed documents as well as the identification of similarities between requirements in specifications of software

products for the same application domain. The ORE (Ontology based Requirements Elicitation) approach by Kaiya and Saeki [28] creates a domain ontology which serves as domain knowledge. This domain knowledge can be used to infer on the incompleteness as well as the inconsistency of requirements definitions, i.e. both methods address the correctness of a requirements specification. Kaindl et al. [29] reuse requirements based on software use cases that are stored in a knowledge base and apply similarity metrics to new use cases, which can obviously speed up requirements elicitation and thus reduce resource input.

A fundamental prerequisite for the reuse of requirements or knowledge related to requirements is the availability of a corresponding knowledge base. Drawing on concepts from the knowledge management domain, a distinction between *retrieved knowledge* and *imported knowledge* is possible [30]. While the former is extracted from the analyst's interaction with the system, the latter represents previously existing knowledge that is available in an appropriate format to be uploaded to the knowledge base. As an example, the SIREN (Simple REuse of software requiremeNts) method used by Toval et al. [31] draws on MAGERIT, a governmental information systems risk analysis and management method, to derive reusable security requirements. These requirements form the basis of the knowledge base, which is considered as an evolutionary product. It will be extended by new requirements found in the development of a system, while old requirements are modified in order to improve their quality. This ‘repository improvement’ [31] can be considered as retrieved knowledge. Accordingly, the identified works are categorized to reuse either 1) no knowledge 2) imported knowledge or 3) retrieved knowledge.

2.2. Evaluation perspective

As described earlier, we are aiming to describe the state of the art in automated requirements elicitation from both design and evaluation perspectives. Evaluation approaches as well as evaluation concepts and measures are described in the following sections.

2.2.1. Evaluation approaches

Hevner et al. [32] distinguish between two experimental design evaluation methods: A *controlled experiment* involves studying the presented method or artifact in a controlled environment, which can be done e.g. by comparing the performance of an analyst using the artifact with the performance of an analyst devoid its support. In contrast, a *simulation* comprises the execution of the artifact with test data [32]. A performance evaluation based on a simulation is possible by comparing the artifact's output to a gold standard set of requirements, which is the output created manually by an expert or a group of experts.

Additionally to experiment evaluations, we differentiated two further types of evaluations: A mere demonstration of the presented artifact or method, e.g. by an application to a real-world example without data collection and analysis is classified as a *proof of concept* in the following, while an evaluation in practice, e.g. in an industrial environment, will be denoted as a *case study*. Accordingly, the identified works have been categorized to evaluate their approaches either by (1) a controlled experiment (2) a simulation (3) a proof of concept or (4) a case study.

2.2.2. Evaluation concepts and measures

To evaluate the effectiveness of automated requirements elicitation, the assessment of the completeness and correctness of the identified requirements is a common practice. *Completeness* ensures that all the information required for a problem definition, i.e. all properties that are desired to hold true, are found within the specification [33]. The *correctness* of a requirements specification is determined by the included share of requirements which

⁶ See Davis et al. [74] for a comprehensive review.

match existing needs; IEEE [34] classifies a requirements specification as correct “if, and only if, every requirement stated therein is one that the software shall meet”.

An operationalization of these constructs is possible by drawing on metrics from the information retrieval domain, specifically precision and recall [35]. *Recall* is defined as the proportion of relevant items that are actually retrieved in answer to a search query and is very commonly used as a measure for completeness [21,36,37]. *Precision* is the proportion of retrieved items that are relevant to the query and is often used as a measure for correctness, usually in combination with recall. Based on the above mentioned gold standard, to achieve a high recall, the tool aims to find as much relevant requirements in the documents as possible. Additionally, to achieve a high precision, the tool strives to keep the proportion of incorrectly specified requirements low.

Improving recall and precision at the same time is a challenge, as maximizing the number of retrieved requirements to improve recall is often done at the cost of also retrieving more irrelevant items, which reduces precision. Trading off precision for recall or vice versa, one might argue that for automatic elicitation, recall is the more important measure of both, as errors of commission are easier to correct than errors of omission [17]. While an omitted requirement needs to be identified within a potentially longer source document, requiring significant time for manual searching, a wrongly identified document can easily be deleted from the list of the all identified requirements. This requires however, that the resulting list of requirements is significantly shorter than the source document. Therefore, recall and precision are sometimes complemented with a third measure describing the summarization provided by the tool. Summarization measures the volume of the tool's output in relation to the input document size. Tools providing a high level of summarization simplify manual corrections of automatically identified requirements as the analyst can concentrate on reviewing the relatively short output of the tool in contrast to its longer input document. Particularly for abstraction identification tools, summarization plays an important role, as this type of tools aims at distilling the key abstractions of an initially long document. We subsumed the concept summarization under the category “Other Concepts” together with further concepts which only appeared in very few works.

In addition to measures for requirements quality, which represent the outcome of the elicitation process, it is also worthwhile to observe the process leading to this outcome. In various works, elicitation *efficiency* is assessed additionally to quality aspects [21,36,37]. Elicitation efficiency can be measured by the *time* required to transform an unstructured input document to a set of structured requirements. In the case of tools supporting semi-automatic elicitation, this time period can be split into two phases: the automation phase and the manual phase. While the duration of the automation phase is determined by the runtime of the automation algorithm, the duration of the manual phase represents the time for manual corrections of the algorithm's findings. It can be argued that the duration of the automation phase is less critical than the duration for manual adaptations, as the automation can run in a background job without absorbing the analyst's time. In contrast, the time for manual adaptations should be observed critically, especially in evaluations which compare automated with manual approaches.

In summary, we believe that a holistic evaluation of tool effectiveness requires a consideration of both aspects (elicitation quality and efficiency).

3. Methodology

To answer the research question, existing literature was analyzed performing a *systematic review* as proposed by Kitchenham

[38]. Before the actual review was conducted, it was planned by establishing a review protocol based on the research question introduced in the first section. This review protocol contained the search strategy, inclusion and exclusion criteria in each step of the search, and the methodology for data extraction and synthesis. The implementation of the review protocol, which guided every step in the search process to ensure rigor and transparency, is reported in the following. Drawing on the experiences reported by Brereton et al. [39], the seminal work of Goldin and Berry [18] was chosen as a natural starting point for the literature analysis. Extending the time frame by 5 years, the search was restricted to the period from 01/1992 to 03/2012 for all mentioned publications and databases. All database queries were made during the second quarter of 2012 and a full-text search was employed where possible.

To identify existing evidence as completely as possible, different data sources were taken into account. A necessary prerequisite was the definition of a tripartite search string. The first part of the search string includes different terms that are used as synonyms for requirements elicitation in the literature. Within the broad range of requirements engineering processes, requirements elicitation represents the focus of this article. Papers covering automated tool support for other requirements engineering processes (e.g., requirements negotiation, validation or management) were only considered if their authors explicitly created a reference (e.g., proposed an alternative usage of their tool) to requirements elicitation. The second part represents the technological concepts of interest, i.e. (semi-)automation and reuse, to select papers which describe the type of tools being focused in the review. Finally, the third part supported the selection of empirically evaluated works, particularly through experiments, case studies or field studies. Table 1 shows the composition of the applied search string. Terms within a row are connected by OR-operators, while the different parts of the search strings are connected by AND-operators. This results in the query string (“*requirements elicitation*” OR “*requirements determination*” OR “*task elicitation*” OR “*requirements gathering*” OR “*requirements discovery*”) AND (*automat** OR *reuse*) AND (*study* OR *experiment*)).

As high quality publications are most likely to be found in leading journals or conferences, first a key word search was employed in the contents of the Requirements Engineering journal, the IEEE Transactions on Software Engineering and the Empirical Software Engineering journal. Beyond this journal-specific search, two major databases for publications in the CS domain, namely IEEE Xplore and the ACM Digital Library, were queried. For the former, the analysis was limited to the first 200 out of over 1100 hits, as the search results are ranked by relevance and no relevant works were identified following the first 100 hits. Furthermore, the key word search was extended to the AIS Senior Scholars' Basket of Journals,⁷ to additionally capture high quality RE works from the Information Systems (IS) field. Similarly to the CS databases, two major IS databases, namely EBSCOhost and ProQuest, were queried with the same search string to maximize the coverage of relevant publications. The search was limited to peer-reviewed journal articles, and the journals analyzed earlier were excluded from the search. To include recent findings, selected conference proceedings with relevance to RE were also taken into account. This covered the IEEE International Requirements Engineering Conference, the ACM/IEEE International Conference on Software Engineering, and the ACM SIGCHI Conference on Human Factors in Computing Systems. As no database search was possible in the proceedings of the REFSQ before 2007, a manual analysis of the conference summaries was employed. The keyword-based search resulted in an initial result set of 1.126 papers.

⁷ <http://home.aisnet.org/displaycommon.cfm?an=1&subarticlenbr=346>.

Table 1
Table of search terms.

AND	OR
	"requirements elicitation", "requirements determination", "task elicitation", "requirements gathering", "requirements discovery"
	automat*, reuse
	study, experiment

Table 2
Classification of included publications.

Type	Included publications	Number of publications
Abstraction identification	Flores [46] Gacitua et al. [41] Goldin and Berry [18] Kof [43] Laukaitis and Vasilecas [45] Omoronyia et al. [44] Sawyer et al. [42]	7 (19.4%)
Requirements model generation	Alkhader et al. [47] Ambriola [11] Harmain and Gaizauskas [48] Kiyavitskaya and Zannone [36] Körner and Gelhausen [49] Mich [50] Mich and Garigliano [51] Pérez-González and Kalita [52] Pérez-González et al. [53] Sampaio et al. [37] Schneider et al. [54] Seresht et al. [55]	12 (33.3%)
Requirements quality analysis	Cybulski and Reed [27] Cybulski and Reed [56] Kaindl et al. [29] Natt och Dag et al. [57] Natt och Dag et al. [58] Park et al. [59]	6 (16.7%)
Requirements identification	Ankori [60] Casamayor et al. [61] Castro-Herrera et al. [62] Celand-Huang et al. [21] Kaiya and Saeki [28] John and Dörr [63] Lee and Bryant [64] Rago et al. [65] Shibaoka et al. [66] Toval et al. [31] Vlas and Robinson [24]	11 (30.6%)
Total		36

The titles and abstracts of the articles included in the initial search results were then examined in order to evaluate whether inclusion was warranted. At this point, a publication was considered relevant if a focus on requirements elicitation and (semi-)automation or the reuse of requirements or knowledge related to requirements was visible from the title and abstract. This analysis reduced the corpus of relevant works to 49 papers, which were reviewed in detail to assess if they contributed to answering the research question. Foremost, the approach should describe support for the conversion of unstructured or semi-structured information to structured requirements, where the source

information is given as unrestricted or restricted natural language. Additionally, at least a basic evaluation of tool effectiveness has to be present. This includes empirical findings concerning the effect on the correctness of elicited requirements or the resource input necessary for an elicitation, as well as justified reasoning on the expected impact. This process step led to the exclusion of 20 publications and a remaining set of 29 papers.

Following the recommendations by Webster and Watson [40], the citations of the articles or conference proceedings found in the remaining set were assessed to find more relevant sources. A forward search was employed as well, using the *Web of Knowledge*⁸ database to find articles citing the respective publication. The forward and backward search was limited to journal articles and conference proceedings due to their review process before publication. Furthermore, commercial tools were referred to in different publications, but were excluded from the analysis as only published scientific results were of interest. The forward/backward search resulted in seven additional relevant papers and a final result set of 36 works. A detailed overview of the number of publications in each process step is given in Fig. 2.

Although the resulting corpus of papers provides an overview of relevant works in the field and its sub-fields, it is difficult to recognize how this research field developed and to retrace former discourse in the field. This information could provide additional insights about unleveraged research potential and further opportunities for future research. Therefore a citation analysis was performed within the corpus of the remaining 36 works. For this purpose, the references in each of the works were analyzed for citations of other works within the corpus, resulting in a 36 * 36 matrix of cross-references. The resulting matrix contained 11 works which were not referenced to other works of the corpus. The remaining 25 papers and their links through references were then depicted in a comprehensive graph providing hints about both the development of the field as well as former discourse.

4. Results

As described in the methodology section, the final result set of the systematic review consisted of 36 works, which were subsequently categorized according to the analysis framework presented previously. Concerning the design perspective, the relevant publications were grouped according to the introduced four tool categories. Furthermore, each tool was classified by its degree of automation and its approach to generate knowledge for the reuse of requirements or requirements knowledge. For the evaluation perspective, the chosen evaluation approach as well as the included concepts and measures were assessed. An overview of the classification, the included technological concepts and the performance evaluation is given in Fig. 3. In the following, the main findings with regard to the research question are summarized.

4.1. Results per tool category

Table 2 depicts the results per tool category. The generation of domain knowledge for the purpose of requirements elicitation can be supported by *abstraction identification* automation approaches, where three tools address the challenge of reducing input documents to their abstractions as a starting point for the generation of requirements [18,41,42]. In contrast to abstractions, not only important concepts within a domain, but also the relations among these concepts are modeled in domain ontologies. Four of the included publications specifically address the benefit this additional information provides for requirements elicitation,

⁸ <http://apps.webofknowledge.com/>.

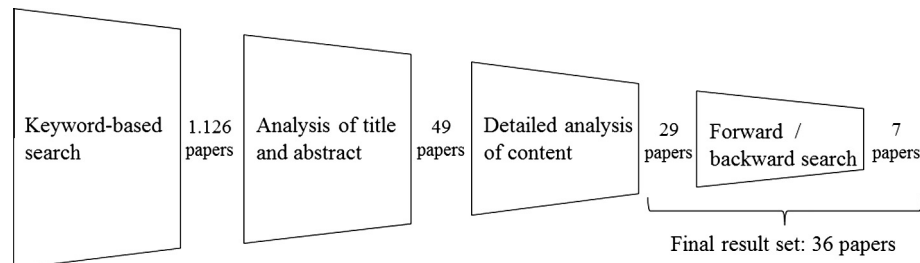


Fig. 2. Number of publications in each process step.

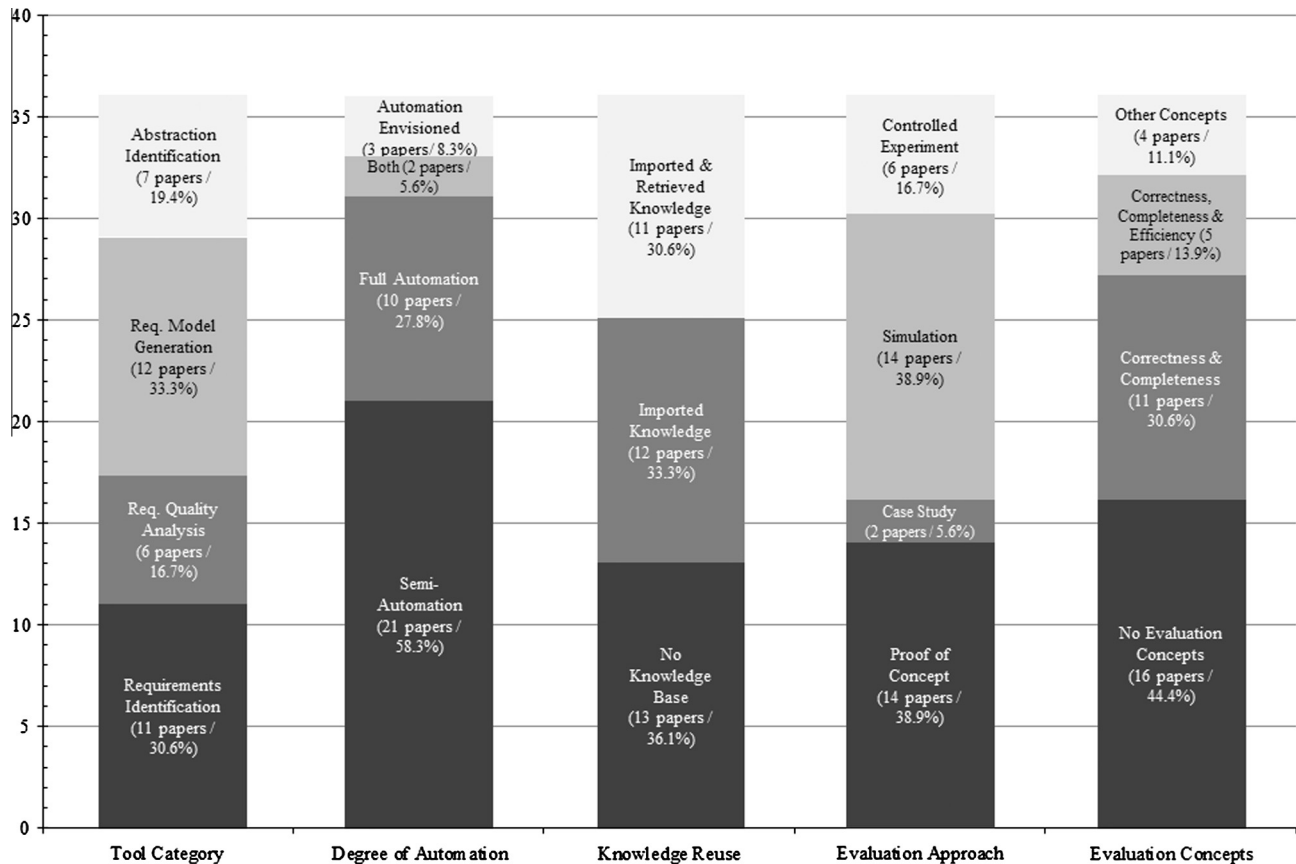


Fig. 3. Overview of included publications.

and present tools and methods to generate domain ontologies fostering requirements elicitation [43–46].

The elicitation of requirements is often followed by a phase where a graphical representation of these requirements, i.e. a requirements model, is created, which fosters both the discussion of requirements with stakeholders and the implementation phase [67]. Several *model generation* artifacts that integrate the identification of requirements in unstructured or semi-structured source documents and their conversion into requirements models have been presented in the literature. Almost all of them (9 out of 12 publications) draw on UML models to model real-world concepts. Besides that, a modeling language specifically tailored to security requirements [36], and a proprietary object-oriented modeling language [50,51] can be found.

Representing the group of *requirements quality analysis*, six publications have been included in the analysis. Three of those identify matching existing requirements based on statements of customer needs [27,29,56], while the three remaining ones address

similarity analysis for requirements [57–59]. The latter ones already lean towards requirements validation, however requirements elicitation may benefit from this similarity analysis, too. As an example, in case a stakeholder suggests a new requirement, it can be assessed if the same or a similar requirement has already been proposed earlier. This may speed up the elicitation process, as existing requirements can be reused.

Finally, tools for *requirements identification* aim directly at providing support for the elicitation of structured requirements from unstructured or semi-structured sources of information by drawing on (semi-)automation, or reuse of requirements respectively knowledge related to requirements. The required input varies and is not always clearly defined, but mostly comprises an informal requirements document in natural language or a corpus of statements on expected features collected from the stakeholders. 7 out of 11 publications present approaches to identify requirements in the input documents [28,31,60,62–64,66], while 4 out of 11 publications also aim at classifying the identified

requirements according to different dimensions [21,24,61,65]. This includes the distinction between functional and non-functional requirements as well as the distinction between different types of non-functional requirements.

4.2. Degree of automation

Table 3 depicts the results per degree of automation and tool category. While the majority of the works summarized in this literature review describe tools that have already been implemented, the papers by Toval et al. [31], John and Dörr [63] and also Kaiya and Saeki [28] focus on the conceptualization of tools to be built. However, they emphasize (semi-)automation as a necessary means to perform the presented method efficiently, and provide the methodological foundation for tool development.

The majority (58.3%) of artifacts presented in the included publications are semi-automated, where the required, or possible, degree of interaction between the human analyst and the tool varies. First, a manipulation of the input data may be necessary. Lee and Bryant [64] and Seresht et al. [55] suggest a manual preprocessing of the input natural language document, e.g. to remove errors in spelling and grammar or ambiguities in the text. Körner and Gelhausen [49] require the analyst to annotate the input natural language document with semantic information, making it semi-structured. Second, the analyst may be required to perform tasks which are not suitable for automation, but necessary as an intermediate step in the tool-assisted process. As an example, Kof's [43] approach to building a domain ontology requires the analyst to manually identify intersections between (automatically built) term clusters, as it is impossible for an automated tool to decide which concept clusters are meaningful. Third, various means may be provided to interact with the artifact's output. Ambriola [11] as well as Mich [50] allow the analyst to manipulate graphical models produced by the tool, e.g. by varying the level of detail. Cleland-Huang et al. [21] as well as Casamayor et al. [61] use the analyst's feedback on requirements classified by their artifact to enhance their classification algorithm.

A less extensive group of approaches (27.8%) draws on full automation. However, none of the authors presenting a fully automated artifact expect that the tool may entirely replace manual elicitation by a requirements engineer. As explained earlier, the output of fully automated artifacts is usually presented to a human analyst for verify at least part of the output. Finally, Harmain and Gaizauskas [48] present two versions of their artifact, which can either

build a requirements model semi-automatically in interaction with the analyst, or produce a model fully automatically.

4.3. Reuse of requirements and requirements knowledge

In Table 4 the results are depicted per type of requirements reuse and tool category. More than one third (36.1%) of the analyzed approaches do not intend to reuse requirements or knowledge related to requirements. Not surprising, this especially applies to abstraction identification tools, as their aim is the formation of knowledge which may later be reused, e.g. by tools for the identification of requirements [43]. Thus, no knowledge may be available beforehand. Other tools do not require knowledge reuse as they draw on domain-agnostic, general techniques. As an example, Harmain and Gaizauskas [48] intentionally omit the use of a domain knowledge base, which is laborious to build, in favor of domain-agnostic natural language processing algorithms. Similarly, Natt och Dag et al. [57] do not have to draw on existing knowledge to analyze the similarity of continuously arriving requirements from different sources, as they use general information retrieval techniques.

Still, the majority of model generation tools (8 out of 12) make use of existing knowledge to identify requirements contained in natural language documents. Similarly, most of the requirements identification tools (9 out of 11) use existing knowledge to create requirements models. The reuse of requirements-related knowledge is widely established. It may be represented in domain knowledge bases, as Ambriola [11] or Liu and Lin [68] demonstrate, or in domain ontologies, which are used e.g. by Kaiya and Saeki [28] or Shibaoka et al. [66]. Besides specific domains, knowledge related to requirements can also cover domain spanning aspects as e.g. non-functional requirements in general [24,61] or security requirements [36,54]. In contrast to the use of requirements-related knowledge, the explicit reuse of existing requirements specifications is less common, and the expected benefits for the overall software development process are emphasized in contrast to the benefits for requirements elicitation [27,31,56].

Many authors (30.6%) use an imported knowledge base but do not regard the imported knowledge as a final, static product rather than aiming at a refinement of the knowledge base by incorporating retrieved knowledge. This can have different forms. Cleland-Huang et al. [21] iteratively train their non-functional requirements classifier based on the analyst's feedback. Kaiya and Saeki [28] consider a refinement of the domain ontology drawing on

Table 3
Degree of automation.

Type	Automation envisioned	Semi-automation	Full automation	Both variants	Total
Abstraction identification	0	4	3	0	7 (19.4%)
Req. model generation	0	7	4	1	12 (33.3%)
Req. quality analysis	0	4	2	0	6 (16.7%)
Requirements identification	3	6	1	1	11 (30.6%)
Total	3 (8.3%)	21 (58.3%)	10 (27.8%)	2 (5.6%)	36

Table 4
Reuse of requirements and requirements knowledge.

Type	No knowledge base	Imported knowledge	Retrieved knowledge	Imported and retrieved knowledge	Total
Abstraction identification	5	2	0	0	7 (19.4%)
Req. model generation	4	4	0	4	12 (33.3%)
Req. quality analysis	2	1	0	3	6 (16.7%)
Requirements identification	2	5	0	4	11 (30.6%)
Total	13 (36.1%)	12 (33.3%)	0 (0.0%)	11 (30.6%)	36

Table 5
Evaluation approaches.

Type	Proof of concept	Case study	Simulation	Controlled experiment	Total
Abstraction identification	3	0	4	0	7 (19.4%)
Req. model generation	5	1	3	3	12 (33.3%)
Req. quality analysis	3	0	2	1	6 (16.7%)
Requirements identification	3	1	5	2	11 (30.6%)
Total	14 (38.9%)	2 (5.6%)	14 (38.9%)	6 (16.7%)	36

Table 6
Evaluation concepts and measures.

Type	No evaluation measures	Correctness and completeness	Correctness, completeness and efficiency ^a	Other concepts	Total
Abstraction identification	3	2	0	2	7 (19.4%)
Req. model generation	6	2	4	0	12 (33.3%)
Req. quality analysis	3	2	1	0	6 (16.7%)
Requirements identification	4	5	0	2	11 (30.6%)
Total	16 (44.4%)	11 (30.6%)	5 (13.9%)	4 (11.1%)	36

^a Assignments to the groups “correctness, completeness and efficiency” and “correctness and completeness” are mutually exclusive.

the information extracted from the requirements statements, thus incorporate retrieved knowledge. Still, imported knowledge is always used as a foundation, and no approach solely draws on retrieved knowledge.

4.4. Evaluation approaches

Table 5 depicts the results classified by evaluation approach and tool category. More than one third (38.9%) of authors solely draw on a proof of concept to demonstrate the applicability of their approach. All of these publications have in common that no form of data collection or analysis is mentioned or included, impeding a profound assessment of their works. The extent of the proofs of concept varies; while some authors use small, self-contained examples to explain the method presented in their publications, as e.g. [60], others use multiple extensive examples based on data from industrial practice, as e.g. [43]. Two publications include reports of case studies, i.e. an evaluation of the respective artifact for a longer period of time in an industrial environment [11,31].

A performance evaluation based on a reference set of requirements is performed in an equally large group (38.9%) of the relevant publications, which include a simulation for performance evaluation. Extensive empirical validation is performed e.g. by Cleland-Huang et al. [21], who evaluate their approach based on 30 requirements specifications as well as a large industrial free-form requirements document and compare their artifact's output to a manually created reference classification. Also Goldin and Berry [18] and Gacitua et al. [41] perform a thorough empirical validation, comparing the tool output with a set of reference abstractions compiled through human effort. In the former work [18], a comparison of the tool's output with manual elicitation showed, that it took three humans 1 month to achieve a result which was even worse than the tool's output after 3 h of processing.

Surprisingly few (16.7%) authors actually compare the performance an analyst is able to achieve using their artifact to the performance that can be achieved by purely manual effort, i.e. without drawing on the presented approach, in a controlled experiment. Sample sizes are in general small, and range from two participants in [37] to twelve test subjects in the experiment reported in [69].⁹

⁹ An experiment evaluating the artifact presented in [50,51] is reported in [69] and has been retrieved in the forward search for publications including references to the former.

4.5. Evaluation concepts and measures

In Table 6 the results are classified by evaluation concept and tool category. Sixteen (44.4%) publications do not include any evaluation measure, which applies, without exception, to the ‘proof-of-concept’ and ‘case study’ evaluation approaches. Another large group (30.6%) of authors applies metrics to measure the correctness and completeness of requirements elicited using their approach. As outlined above, most authors draw on the common information retrieval metrics precision and recall in the operationalization of these concepts. The calculation of precision and recall is usually done based on a manually compiled reference set of requirements.

Both correctness and completeness are integrated into one measure by Natt och Dag et al. [57], who use *Accuracy* to give an indication on how well requirements duplicates are identified in a natural language document. They integrate true positives (actual duplicates) and true negatives (different requirements identified as non-duplicates) into one comprehensive measure. Kaiya and Saeki [28] introduce four metrics to measure the quality of requirements elicited based on an ontology, among which *Correctness* measures the rate of appropriate requirements, and *Completeness* measures the rate of probably missing requirements.

Only five (13.9%) publications include an efficiency evaluation of the presented artifact. Three of those add the time spent on the elicitation of requirements as a dependent variable complementing precision and recall in a controlled experiment [37,69], while Kiyavitskaya and Zannone [36] compare the time needed to produce a reference set of requirements to the time necessary to elicit requirements using the presented approach. Two additional publications include time in their analysis, but do not give a reference that allows for a comparison of efficiency. Cleland-Huang et al. [21] mention the time necessary to manually classify their sample set of requirements without including an analysis of the time using their approach. In contrast, Vlas and Robinson [24] state the time required for their artifact to parse texts, but do not include the effort for a manual analysis.

Besides, different non-standard evaluation measures can be found in four publications, as those measures are most suitable for the respective task intended. In their work on abstraction identification, Goldin and Berry [18] evaluate the effectiveness of their artifact by using assessing the degree of *summarization* it provides. As Kiyavitskaya and Zannone [36], they also compare the effort to

produce the reference set of requirements to their artifact's runtime. Castro-Herrera et al. [62] use the *Normalized Mutual Information* metric to assess the quality of clusters of requirements statements. A high score of this metric indicates that the clustering predefined by a human analyst would resemble the output of the automated clustering. Shibaoka et al. [66] simply perform a numerical comparison of requirements identified using their approach to values from the literature, and Laukaitis and Vasilecas [45] use the proportion of correctly identified concepts as metrics for comparison.

4.6. Result of the citation analysis

As mentioned in the methodology section, from the corpus of 36 relevant papers, 25 papers were referenced to other works of the corpus. These 25 publications and their links through references are depicted in Fig. 4. The tip of each arrow points to the paper being referenced.

The figure allows several interesting observations. First, it can be seen that single papers served as seminal papers inspiring further research of the entire field (e.g. [18]) or one sub-field (e.g. [50] for “Requirements Model Generation”). Second, it can be observed that (because of these seminal papers) some sub-fields strongly influenced other fields (e.g. “Abstraction Identification”), while works from other sub-fields primarily cited works within their own domain (e.g. “Requirements Model Generation”). In general, apart from [18], only very few works are referenced across sub-fields. Furthermore cross-referencing can be mainly observed for more recent works, while earlier works mainly cited within their own sub-field. Third, indications for a temporal sequence of

research activities can be observed, for example some works within the “Requirements Quality Analysis” and “Abstraction Identification” sub-fields apparently served as precursors of “Requirements Identification” works. Fourth, it is worthwhile noticing the absence of central papers containing both a large number of inbound and outbound references. Strongly related papers within the corpus either serve as seminal papers, which are frequently referenced by later works or synthesizing papers, which reference a large number of previous works.

5. Discussion and propositions for future research

Results of our research show, that additionally to the three types of automated elicitation tools described by Berry et al. [17], a fourth type dealing with the mere identification of requirements can be differentiated. Tools of this fourth type do not transform initial documents into models yet, but focus on the first identification and categorization of requirements statements. Therefore, in the elicitation process they can be positioned between abstraction identification tools and requirements modeling tools. In the following, we will discuss observations of our analysis along the depicted framework, point out problems and gaps and propose future research activities to address them.

5.1. Degree of automation

In our research framework, we differentiated semi-automated and fully automated elicitation approaches. A significant percentage (27.8%) of the identified systems only works in a fully automated mode. While these approaches can take a decent amount

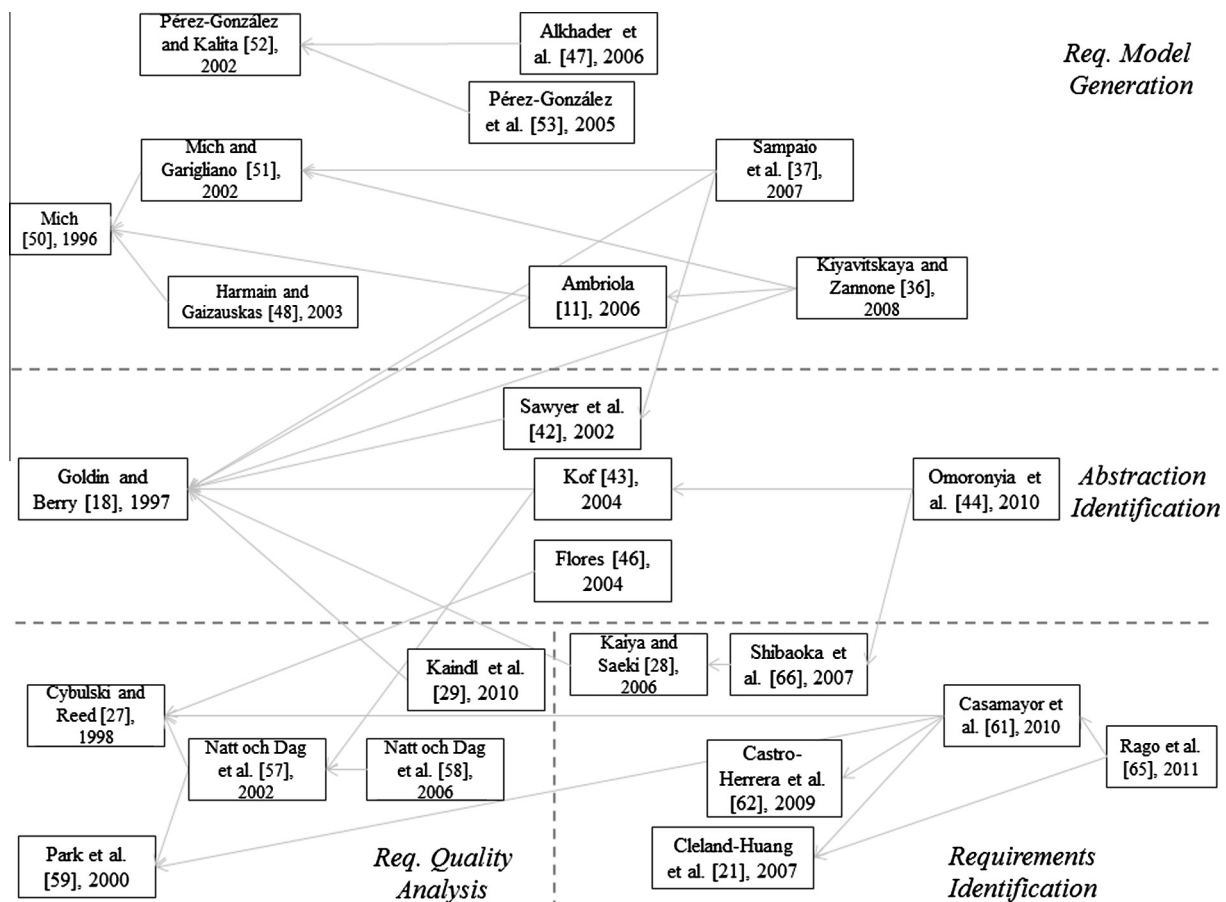


Fig. 4. Results of citation analysis within corpus.

of work from the requirements engineer, tools which do not adequately support manual adaptations of the automatism's findings can support only (the first) part of the elicitation process. This is due to the circumstance, that automated requirements elicitation results in recall or precision rates not very close to 100% [21,36,37]. Natural language and its inherent ambiguity and inconsistency forces requirements engineers to check and sometimes manually rework results of the automatism. This includes correcting mistakes, extending or limiting the identified text passages or simply adding requirements that were overlooked. Consequently, the automatism needs to be complemented with functionality supporting manual elicitation.

While a lot of effort has been put into the optimization of the automatism functionality, little attention has been paid to provide efficient manual elicitation capabilities. The design of highly usable interfaces that allow easy and fast corrections of the tools' initial findings could provide a valuable means to further improve elicitation efficiency. In the same context, the exploration of alternative interaction paradigms other than a purely sequential approach (first automatic recommendations, then subsequent manual verification) could be researched. For example, it could be interesting to investigate tools that split the identification task (manual) and the categorization task (automatic) or tools that generate recommendations in a more interactive way, e.g. reacting on manual corrections of former recommendations.

5.2. Reuse of requirements and requirements knowledge

We differentiated approaches which do not require any knowledge base, tools that use imported knowledge and tools which use retrieved knowledge. Disregarding abstraction identification tools, which by definition have the purpose to create and not to reuse knowledge, the majority (72.4%) of the remaining tools depend on the pre-existence of a knowledge base to be able to execute the automation process. However, the creation and maintenance of a knowledge base can represent a time-consuming task in itself [11]. Due to the domain-specific context of a requirements document, a knowledge base needs to be continuously extended and adapted to be able to identify a significant amount of requirements. Looking at the overall elicitation effectiveness, these efforts have to be included and can put the added value of automatic elicitation into question. To limit these efforts, reuse of requirements knowledge is a promising approach. Reuse has shown to be a valuable concept to reduce efforts in other areas of software development and has been an important topic in software engineering research and practice almost as long as the existence of the field itself [70]. It has been a core element of numerous implementation approaches: from the early ideas to reuse “mass produced software components” [71], via the reuse of objects through inheritance in object-oriented methodology [72] to the notion of product lines, defining a set of systems which share components based on a common architecture [73].

To foster reuse in automated requirements elicitation, we suggest considering the usage of different types of knowledge. While imported knowledge can provide a first starting point for knowledge reuse, it needs to be created manually and therefore can be too static to adapt to new, domain-specific requirements documents. Therefore different techniques have been proposed to feed elicitation results back into the knowledge base through the usage process itself [21,28]. We refer to this knowledge as “retrieved knowledge” (in contrast to imported knowledge which is usually provided up-front). For all described tool categories except those of the abstraction identification type we therefore suggest for future research to utilize both imported as well as retrieved knowledge to increase the quality of automatically elicited

requirements on the one hand side, and limit knowledge creation efforts on the other one.

5.3. Evaluation approaches

The identified research works were primarily evaluated via simulations, case studies, proofs-of-concept or combinations of these approaches. While these evaluations allow precise measurements of absolute quality criteria, they do not allow a comparison to the as-is situation of manual elicitation or a comparison to other automated approaches. Consequently, the question of whether the tools really improve requirements quality and elicitation efficiency cannot always be answered. Unlike first intuition would tell us, even efficiently working automated elicitation does not necessarily outperform manual elicitation. As described earlier, due to the ambiguity and inconsistency of natural language documents, results of automated elicitation in most cases require manual rework. Therefore, even automated approaches resulting in high (but not 100%) initial recall and precision might generate larger total efforts as manual elicitation if times for rework are also taken into account. Comparative evaluation methods like experiments or case studies contrasting both approaches can address this issue and increase explanatory power. Consequently, we propose for future research on tool-supported elicitation to put a stronger focus on comparative evaluations contrasting the own, new approach with former ones. Evaluations may be based on objective measurements in the context of lab or field experiments or subjective measurements in the context of case or field studies.

For example in an experiment, an objective measurement of recall, precision and effort is possible. Through multiple treatments or tasks (depending on the design of the experiment) the proposed approach can be compared to an alternative (manual or semi-automatic) approach. By providing the utilized input files as a reference data set, subsequent evaluations can be based on the same prerequisites. In contrast to an experiment, case studies usually cannot be replicated in a similar project context. Therefore reference values for an objective measurement can hardly be obtained. However, the mentioned measures can be assessed through subjective measurement (e.g. through requirements engineers filling out a post-project survey in which the new approach is compared to the former one). Despite these general suggestions for comparative evaluations, it should be noted, however, that a valid, comparative evaluation depends both on the selection of an appropriate evaluation method as well as its rigorous application. Furthermore, although a summative evaluation (as proposed here) can be very valuable to assess tool efficiency at the end of a research project, it might be less helpful during the design of the tool itself (in this phase of a project formative feedback could be more important).

5.4. Evaluation concepts and measures

The evaluations of the identified works concentrated on the assessment of requirements quality, especially completeness and correctness (measured by recall and precision). In line with the argumentation in the previous section, we suggest that although these measures give clear indications for the effectiveness of the investigated tools, a high resulting requirements quality is no proof, that a new elicitation approach is superior to a previous one (e.g., manual elicitation), as improved elicitation quality could come at the cost of higher total efforts. Therefore the measurement of elicitation efficiency (in addition to elicitation quality) should be considered. However, an according measurement potentially increases the complexity of the evaluation and raises multiple questions which need to be answered in the context of each research project individually:

Table 7

Classification of included publications per journal/conference.

Journal/conference	Publication type	Number of publications
Requirements Engineering	Journal	6 (16.7%)
Automated Software Engineering	Journal	4 (11.1%)
International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ)	Conference	4 (11.1%)
Information and Software Technology	Journal	2 (5.6%)
IEEE International Requirements Engineering Conference	Conference	2 (5.6%)
Object-Oriented Programming, Systems, Languages & Applications (OOPSLA)	Conference	2 (5.6%)
Others	Journals and conferences	16 (44.4%)
Total		36

Which time frame should be measured? Depending on which functionality of a tool should be evaluated, one could argue to either measure 1) only the time of the automation process 2) only the time for the manual adaption of the automation results or 3) both times. How can the measurement of efficiency be organized? Unlike requirements quality, which can be assessed at the end of the elicitation process, elicitation efficiency needs to be continuously tracked. While this is a manageable task in a lab environment, it can become very challenging in a real life scenario (e.g., a case study in a company), in which requirements elicitation might be continuously interrupted by other activities. Does the measurement of efficiency affect requirements quality? Whenever participants of an evaluation are aware of the fact that both requirements quality and elicitation efficiency are measured, they might trade off each against the other. Consequently, a measurement of elicitation efficiency could corrupt the measurement results for requirements quality, as each participant of an evaluation could set individual priorities (either focus on quality or efficiency).

In spite of these challenges, we are confident that a carefully conducted evaluation combining quality and efficiency measures can raise the explanatory power of automated requirements elicitation evaluations in comparison to a pure, quality-centered approach. We therefore propose for future research to consider combined measurements of elicitation quality and efficiency and hereby enable a more holistic comparison of elicitation approaches.

5.5. Discussion of the citation analysis result

Based on our citation analysis, the following proposition for future research can be drawn. The small number of references across the four sub-fields indicates that there is still large potential for synergies and joint research endeavors across sub-fields. For example, surprisingly little results of the “Model Generation” sub-field seem to have been used in current research on “Requirements Identification”, although there is a strong overlap in the first phases of the corresponding processes. Joint research, bringing together scholars from multiple sub-fields of automated requirements elicitation could result in significant progress for the field.

6. Conclusion

This paper aimed to capture the current state of automated requirements elicitation and identify future research avenues in this area. To achieve this, a systematic review of existing literature was conducted. 36 publications were identified as relevant, and analyzed with regard to their tool category, technological concepts and evaluation approaches. Summarizing the observations we made concerning these papers, we identified opportunities for future research and formulated propositions about how to address them.

In order to adequately interpret the implications of our findings, the following limitations of our study need to be considered. Even though a systematic review should ensure a relatively complete census of relevant literature, completeness can never be guaranteed, and every review most likely will miss some relevant articles [40]. While the terminology used in the database query is commonly accepted and used within the scientific community, different terms might have been used to describe relevant methods, which may result in failure to recognize them. Besides, while clear criteria for the assessment of relevance have been established in Sections 3 and 4, the evaluation was based on the judgment and experience of the authors,¹⁰ and other researchers might have judged differently. The same limitation applies to the classification of each paper according to the dimensions and values of our framework. Therefore, two researchers participated in the paper classification, discussing doubts and resolving conflicts when necessary. Finally, the content and structure of the analysis framework itself can only represent an excerpt of interesting characteristics to be investigated. Future research might complement our study, classifying the same or similar sets of papers according to additional dimensions and characteristics.

Regardless of these limitations, we believe that the following contributions can be drawn from our study. First, it derives an analysis framework for works in the area of automated requirements elicitation, going beyond former classifications. Besides the application in this paper, the framework might be used to classify and evaluate future research in this area. Second, the current state of the art in automated requirements elicitation has been depicted based on the derived framework. Providing a broad overview of existing works, this compilation might be useful as a starting point for scholars who are about to research in this area. Third, we formulated concise propositions for future research, based on the identified limitations of existing works and the conducted citation analysis. We hope that our propositions can help to close existing gaps in the design and evaluation of innovative tools, and to inspire joint projects across sub-fields of automated requirements elicitation research.

Acknowledgements

The authors thank Daniel M. Berry and Frederik Ahlemann for their helpful feedback on earlier versions of this paper.

Appendix A.

Table 7 shows how the included publications are distributed across different journals and conferences. Conferences and journals with one single publication were summarized as “Others”.

¹⁰ All three authors, additionally to their research activities, gathered industry experience in software engineering (with an average experience of 5.3 years).

<i>Semi-Automation</i>		<i>Full Automation</i>
Abstraction Identification	Omoronyia et al. [44] Sawyer et al. [42] Kof [43] Laukaitis and Vasilecas [45]	Gacitua et al. [41] Goldin and Berry [18] Flores [46]
Requirements Quality Analysis	Kaindl et al. [29] Cybulski and Reed [27,56]	Natt och Dag et al. [57, 58] Park et al. [59]
Requirements Model Generation	Körner and Gelhausen [49] Mich and Garigiano [51] Alkhader et al. [47] Ambriola [11] Mich [50] Sampaio et al. [37] Schneider et al. [54]	Kiyavitskaya and Zannone [36] Pérez-González and Kalita [52] Pérez-González et al. [53] Seresht et al. [55]
Requirements Identification	Casamayor et al. [61] Ankori [60] Lee and Bryant [64] Castro-Herrera et al. [62] Rago et al. [65] Shibaoka et al. [66]	Vlas and Robinson [24]
Automation Envisioned	Kaiya and Saeki [28] Toval et al. [31] John and Dörr [63]	Cleland-Huang et al. [21] Harmain and Gaizauskas [48] <i>Automation & Semi-Automation</i>

Fig. 5. Classification of included publications per tool category and degree of automation.

<i>No Knowledge</i>	<i>Imported Knowledge</i>	<i>Imported & Retrieved Knowledge</i>
Abstraction Identification Goldin and Berry [18] Gacitua et al. [41] Sawyer et al. [42] Kof [43] Omoronyia et al. [44]	Laukaitis and Vasilecas [45] Flores [46]	
Requirements Quality Analysis Natt och Dag et al. [57, 58]	Park et al. [59]	Cybulski and Reed [27,56] Kaindl et al. [29]
Requirements Model Generation Alkhader et al. [47] Harmain and Gaizauskas [48] Pérez-González and Kalita [52] Pérez-González et al. [53]	Ambriola [11] Kiyavitskaya and Zannone [36] Körner and Gelhausen [49] Seresht et al. [55]	Sampaio et al. [37] Mich [50] Mich and Garigiano [51] Schneider et al. [54]
Requirements Identification Kaiya and Saeki [28] Castro-Herrera et al. [62]	Vlas and Robinson [24] John and Dörr [63] Lee and Bryant [64] Rago et al. [65] Shibaoka et al. [66]	Cleland-Huang et al. [21] Toval et al. [31] Ankori [60] Casamayor et al. [61]

Fig. 6. Classification of included publications per tool category and reuse of requirements knowledge.

Appendix B.

Figs. 5 and 6 show how the included publications have been classified concerning their tool category, degree of automation and reuse of requirements knowledge.

References

- [1] H.F. Hofmann, F. Lehner, Requirements engineering as a success factor in software projects, *IEEE Software* 18 (2001) 58–66.
- [2] T. Hall, S. Beecham, A. Rainer, Requirements problems in twelve software companies: an empirical analysis, *IEEE Software* 149 (2002) 153–160.
- [3] G.J. Browne, M.B. Rogich, An empirical investigation of user requirements elicitation: comparing the effectiveness of prompting techniques, *Journal of Management Information Systems* 17 (2001) 223–249.
- [4] C. Urquhart, Themes in early requirements gathering: the case of the analyst, the client and the student assistance scheme, *Information Technology & People* 12 (1999) 44–70.
- [5] L. Mathiassen, T. Saarinen, T. Tuunanen, M. Rossi, A contingency model for requirements development, *Journal of the AIS* 8 (2007) 569–597.
- [6] I. Sommerville, J. Ransom, An empirical study of industrial requirements engineering process assessment and improvement, *ACM Transactions on Software Engineering and Methodology* 14 (2005) 85–117.

- [7] L. Mich, M. Franch, I.P. Novi, Market research for requirements analysis using linguistic tools, *Requirements Engineering* 9 (2004) 40–56.
- [8] A. Casamayor, D. Godoy, M. Campo, Mining textual requirements to assist architectural software design: a state of the art review, *Artificial Intelligence Review* 38 (3) (2012) 173–191.
- [9] W.M. Wilson, L.H. Rosenberg, L.E. Hyatt, Automated analysis of requirement specifications, in: *Proceedings of the 19th International Conference on Software Engineering (ICSE '97)*, 1997, pp. 161–171.
- [10] W.F. Tichy, S.J. Koerner, Text to software: developing tools to close the gaps in software engineering, in: *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research (FoSER '10)*, 2010, pp. 379–384.
- [11] V. Ambriola, On the systematic analysis of natural language requirements with CIRCE, *Automated Software Engineering* 13 (2006) 107–167.
- [12] J. Huffman Hayes, A. Dekhtyar, S. Sundaram, Text mining for software engineering: how analyst feedback impacts final results, in: *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR '05)*, 2005, pp. 1–5.
- [13] B. Nuseibeh, S. Easterbrook, requirements engineering: a roadmap, in: *Proceedings of the Conference on The Future of, Software Engineering (ICSE'00)*, 2000, pp. 35–46.
- [14] B.H.C. Cheng, J.M. Atlee, Research directions in requirements engineering, in: *Proceedings of the 2007 Future of Software Engineering (FOSE '07)*, 2007, pp. 285–303.
- [15] G.-C. Roman, A taxonomy of current issues in requirements engineering, *Computer* 18 (1985) 14–23.
- [16] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, Springer, Berlin/Heidelberg, 2010.
- [17] D. Berry, R. Gacitua, P. Sawyer, S.F. Tjong, The case for dumb requirements engineering tools, in: *Requirements Engineering: Foundation for Software Quality*, Springer, Berlin/Heidelberg, 2012, pp. 211–217.
- [18] L. Goldin, D.M. Berry, AbstrFinder, A prototype natural language text abstraction finder for use in requirements elicitation, *Automated Software Engineering* 4 (1997) 375–412.
- [19] D. Popescu, S. Rugaber, N. Medvidovic, D. Berry, Reducing ambiguities in requirements specifications via automatically created object-oriented models, in: B. Paech, C. Martell (Eds.), *Innovations for Requirement Analysis, From Stakeholders' Needs to Formal Designs*, Springer, 2008, pp. 103–124.
- [20] F. Fabbri, M. Fusani, S. Gnesi, G. Lami, The linguistic approach to the natural language requirements quality: benefit of the use of an automatic tool, in: *Proceedings of the 26th Annual NASA Goddard Software Engineering Workshop*, 2001, pp. 97–105.
- [21] J. Cleland-Huang, R. Settini, X. Zou, P. Solc, Automated classification of non-functional requirements, *Requirements Engineering* 12 (2007) 103–120.
- [22] D.M. Berry, Natural language and requirements engineering – Nu? in: *International Workshop on Requirements Engineering*, Imperial College, London, UK, 2001.
- [23] R. Parasuraman, T.B. Sheridan, C.D. Wickens, A model for types and levels of human interaction with automation, *IEEE Transactions on Systems, Man and Cybernetics, Part A: Systems and Humans* 30 (2001) 286–297.
- [24] R. Vlas, W.N. Robinson, A rule-based natural language technique for requirements discovery and classification in open-source software development projects, in: *Proceedings of the 44th Hawaii International Conference on System Sciences (HICSS '11)*, 2011, pp. 1–10.
- [25] S. Robertson, Reuse lifecycle: essentials and implementations, in: *International Workshop on Systematic Reuse*, 1996.
- [26] W.B. Frakes, K. Kang, Software reuse research: status and future, *IEEE Transactions on Software Engineering* 31 (2005) 529–537.
- [27] J.L. Cybulski, K. Reed, Requirements classification and reuse: crossing domain boundaries, in: *Software Reuse: Advances in Software Reusability*, Springer, Berlin/Heidelberg, 2000, pp. 190–210.
- [28] H. Kaiya, M. Saeki, Using domain ontology as domain knowledge for requirements elicitation, in: *Proceedings of the 14th IEEE International Requirements Engineering Conference (RE'06)*, 2006, pp. 189–198.
- [29] H. Kaindl, M. Smialek, W. Nowakowski, Case-based reuse with partial requirements specifications, in: *Proceedings of the 18th IEEE International Requirements Engineering Conference (RE '10)*, 2010, pp. 399–400.
- [30] S. Staab, R. Studer, H.-P. Schnurr, Y. Sure, Knowledge processes and ontologies, *IEEE Intelligent Systems* 16 (2001) 26–34.
- [31] A. Toval, J. Nicolás, B. Moros, F. García, Requirements reuse for improving information systems security: a practitioner's approach, *Requirements Engineering* 6 (2002) 205–219.
- [32] A.R. Heyner, S.T. March, J. Park, S. Ram, Design science in information systems research, *MIS Quarterly* 28 (2004) 75–105.
- [33] D. Zowghi, V. Gervasi, On the interplay between consistency, completeness, and correctness in requirements evolution, *Information and Software Technology* 45 (2003) 993–1009.
- [34] IEEE, IEEE Recommended Practice for Software Requirements Specifications. <<http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=720574&userType=inst>> (20.10.98).
- [35] G. Salton, M.J. McGill, *Introduction to Modern Information Retrieval*, McGraw-Hill Inc., 1983.
- [36] N. Kiyavitskaya, N. Zannone, Requirements model generation to support requirements elicitation: the Secure Tropos experience, *Automated Software Engineering* 15 (2008) 149–173.
- [37] A. Sampaio, A. Rashid, R. Chitchyan, P. Rayson, EA-miner: towards automation in aspect-oriented requirements engineering, in: A. Rashid, M. Aksit (Eds.), *Transactions on Aspect-Oriented Software Development III*, Springer, Berlin/Heidelberg, 2007, pp. 4–39.
- [38] B. Kitchenham, *Procedures for Performing Systematic Reviews*, Keele, UK, 2004.
- [39] P. Brereton, B.A. Kitchenham, D. Budgen, M. Turner, M. Khalil, Lessons from applying the systematic literature review process within the software engineering domain, *Journal of Systems and Software* 80 (2007) 571–583.
- [40] J. Webster, R.T. Watson, Analyzing the past to prepare for the future: writing a literature review, *MIS Quarterly* 26 (2002) 13–23.
- [41] R. Gacitua, P. Sawyer, V. Gervasi, Relevance-based abstraction identification: technique and evaluation, *Requirements Engineering* 16 (2011) 251–265.
- [42] P. Sawyer, P. Rayson, R. Garside, REVERE: support for requirements synthesis from documents, *Information Systems Frontiers* 4 (2002) 343–353.
- [43] L. Kof, An application of natural language processing to domain modelling – two case studies, *International Journal on Computer Systems Science Engineering* 20 (2004) 37–52.
- [44] I. Omoronyia, G. Sindre, T. Stålhane, S. Biffl, T. Moser, W. Sunindyo, A domain ontology building process for guiding requirements elicitation, in: R. Wieringa, A. Persson (Eds.), *Requirements Engineering: Foundation for Software Quality*, Springer, Berlin/Heidelberg, 2010, pp. 188–202.
- [45] A. Laukatis, O. Vasilecas, Integrating all stages of information systems development by means of natural language, in: P. Sawyer, B. Paech, P. Heymans (Eds.), *Requirements Engineering: Foundation for Software Quality (REFSQ 2007)*, Trondheim, Norway, 2007, pp. 218–231.
- [46] J.J.G. Flores, Linguistic processing of natural language requirements: the contextual exploration approach, in: *Proceedings of the 10th International Workshop on Requirements Engineering: Foundation for Software, Quality (REFSQ'04)*, 2004, pp. 99–112.
- [47] Y. Alkhader, A. Hudaib, H. Hammo, Experimenting with extracting software requirements using NLP approach, in: *Proceedings of the International Conference on Information and Automation (ICIA 2006)*, 2006, pp. 349–354.
- [48] H.M. Harmain, R.J. Gaizauskas, CM-builder: a natural language-based CASE tool for object-oriented analysis, *Automated Software Engineering* 10 (2003) 157–181.
- [49] S.J. Körner, T. Gelhausen, Improving automatic model creation using ontologies, in: *Proceedings of the Twentieth International Conference on Software Engineering & Knowledge Engineering (SEKE'2008)*, 2008, pp. 691–696.
- [50] L. Mich, NL-OOPS: from natural language to object oriented requirements using the natural language processing system LOLITA, *Natural Language Engineering* 2 (1996) 161–187.
- [51] L. Mich, R. Garigliano, NL-OOPS: a requirements analysis tool based on natural language processing, in: *Data Mining III*, WIT Press, Southampton, UK, 2002, pp. 321–330.
- [52] H.G. Pérez-González, J.K. Kalita, Automatically generating object models from natural language analysis, in: *Companion of the 17th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications - OOPSLA '02*, 2002, pp. 86–87.
- [53] H.G. Pérez-González, J.K. Kalita, A. Salvador, R.S. Wiener, GOOAL: An educational object oriented analysis laboratory, in: *Companion to the 20th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications (OOPSLA '05)*, 2005, pp. 180–181.
- [54] K. Schneider, E. Knauss, S. Houb, S. Islam, J. Jürjens, Enhancing security requirements engineering by organizational learning, *Requirements Engineering* 17 (2012) 35–56.
- [55] S.M. Seresht, O. Ormandjieva, S. Sabra, Automatic conceptual analysis of user requirements with the requirements engineering assistance diagnostic (READ) tool, in: *Proceedings of the Sixth International Conference on Software Engineering Research, Management and Applications*, 2008, pp. 133–142.
- [56] J.L. Cybulski, K. Reed, Computer-assisted analysis and refinement of informal software requirements documents, in: *Proceedings of the 1998 Asia Pacific Software Engineering Conference*, 1998, pp. 128–135.
- [57] J. Natt och Dag, B. Regnell, P. Carlshamre, M. Andersson, J. Karlsson, A feasibility study of automated natural language requirements analysis in market-driven development, *Requirements Engineering* 7 (2002) 20–33.
- [58] J. Natt och Dag, T. Thelin, B. Regnell, An experiment on linguistic tool support for consolidation of requirements from multiple sources in market-driven product development, *Empirical Software Engineering* 11 (2006) 303–329.
- [59] S. Park, H. Kim, Y. Ko, J. Seo, Implementation of an efficient requirements-analysis supporting system using similarity measure techniques, *Information and Software Technology* 42 (2000) 429–438.
- [60] R. Ankori, Automatic requirements elicitation in agile processes, in: *Proceedings of the IEEE International Conference on Software - Science, Technology and, Engineering*, 2005, pp. 101–109.
- [61] A. Casamayor, D. Godoy, M. Campo, Identification of non-functional requirements in textual specifications: a semi-supervised learning approach, *Information and Software Technology* 52 (2010) 436–445.
- [62] C. Castro-Herrera, C. Duan, J. Cleland-Huang, B. Mobasher, A recommender system for requirements elicitation in large-scale software projects, in: *Proceedings of the 2009 ACM Symposium on Applied Computing - SAC '09*, 2009, pp. 1419–1426.
- [63] I. John, J. Dörr, Elicitation of requirements from user documentation, in: *Proceedings of the 9th International Workshop on Requirements Engineering - Foundation of Software, Quality (REFSQ'03)*, 2003, pp. 17–26.
- [64] B.-S. Lee, B.R. Bryant, Automation of software system development using natural language processing and two-level grammar, in: *Lecture Notes in Computer Science*, Springer, Berlin/Heidelberg, 2004, pp. 219–233.

- [65] A. Rago, C. Marcos, J.A. Diaz-Pace, Uncovering quality-attribute concerns in use case specifications via early aspect mining, *Requirements Engineering* (2011) 1–18.
- [66] M. Shibaoka, H. Kaiya, M. Saeki, GOORE: Goal-oriented and ontology driven requirements elicitation method, in: *Advances in Conceptual Modeling – Foundations and Applications*, Springer, Berlin/Heidelberg, 2007, pp. 225–234.
- [67] I. Sommerville, *Software Engineering*, Pearson Education Limited, Edinburgh Gate, Harlow, UK, 2007.
- [68] L. Liu, L. Lin, ARED-CK: an automated requirements elicitation approach based on decision-making with complete knowledge, in: *Proceedings of the First International Workshop on Managing Requirements Knowledge (MARK '08)*, 2008, pp. 47–52.
- [69] N. Kiyavitskaya, N. Zeni, L. Mich, J. Mylopoulos, Experimenting with linguistic tools for conceptual modelling: quality of the models and critical features, in: *Proceedings of the 9th International Conference on Applications of Natural Languages to Information Systems (NLDB 2004)*, 2004, pp. 135–146.
- [70] J. Bosch, *Design and Use of Software Architectures: Adopting and Evolving a Product-line Approach*, Addison-Wesley Professional, 2000.
- [71] M.D. McIlroy, J.M. Buxton, P. Naur, B. Randell, Mass-produced software components, in: *Proceedings of the 1st International Conference on Software Engineering*, Garmisch-Partenkirchen, Germany, 1968, pp. 88–98.
- [72] B. Meyer, *Object-oriented Software Construction*, Prentice Hall, New York, NY, USA, 1988.
- [73] K. Pohl, G. Bockle, F. Van Der Linden, *Software Product Line Engineering*, Springer, Berlin/Heidelberg, 2005.
- [74] A.M. Davis, O. Dieste, A. Hickey, N. Juristo, A.M. Moreno, Effectiveness of requirements elicitation techniques: Empirical results derived from a systematic review., in: *Proceedings of the 14th IEEE International Requirements, Engineering Conference (RE'06)*, 2006, pp. 176–185.